

this function from the Octave prompt.

```
octave:1> hello_octave(1,2,3)
This function has 3 input arguments and 0 output arguments.

octave:2> hello_octave()
This function has 0 input arguments and 0 output arguments.

octave:3> help hello_octave
'hello_octave' is a function from the file /home/r00t/Writings/new/octave/
parts/codes/hello_octave.oct

I am a Help String
Additional help for built-in functions and operators is
available in the on-line version of the manual. Use the command
'doc <topic>' to search the manual index.

Help and information about Octave is also available on the WWW
at http://www.octave.org and via the help@octave.org
mailing list.
```

Let us now analyse our simple Octave extension:

- **octave/oct.h:** This file will usually be in the `/usr/include/octave` directory. It includes other files needed to create Oct-files. In fact, its contents are a number of `#include` statements.
- **DEFUN_DLD:** This is a macro defined in `octave/defundld.h`. It indicates the entry point to the dynamically loaded function. As you can see, it takes four arguments: the function name, the list of arguments, the number of output arguments, and the help string.
- **octave_value_list:** This is a class defined in `octave/oct-obj.h` that acts as a container for passing values to and from dynamically loaded extensions.
- **octave_stdout:** This is an alias that is actually defined by a `#define` in `octave/pager.h` and technically makes use of the Octave pager to display output. The pager is defined in the Octave sources in `pager.cc`.

Points to note:

- **File names:** The extension file name should be the same as the name of the Oct-file, since Octave looks for an Oct-file based on the function name.
- **Multiple functions in one file:** If you want to define more than one function in a single file, just create symbolic links for each of the `function-name.oct` files to the main `.oct` file. To illustrate this, we will add (below) one more function to our previous file, `hello_octave.cc`.
- **Search path:** If you have read Part 4 of this series of articles, you will remember that we discussed the locations where Octave searches for function definitions. The same rules apply here.

Accessing multiple functions via symbolic links

```
/*hello_octave.cc */

#include <octave/oct.h>
```

```
DEFUN_DLD (hello_octave, args, nargout,
          "I am a Help String")
{
    int nargin = args.length ();
    octave_stdout << "This function has " << nargin
        << " input arguments and "
        << nargout << " output arguments.\n";
    return octave_value_list ();
}

DEFUN_DLD (hello_octave_1, args, nargout,
          "I am a Help String")
{
    int nargin = args.length ();
    octave_stdout << "Hello from the second function! function has " << nargin
        << " input arguments and "
        << nargout << " output arguments.\n";
    return octave_value_list ();
}
```

Create the Oct-file as we did earlier. Now, also create a symbolic link for the `hello_octave_1` function:

```
ln -s hello_octave.oct hello_octave_1.oct
```

```
octave:1> help hello_octave
'hello_octave' is a function from the file /home/r00t/Writings/new/octave/
parts/codes/hello_octave.oct
```

```
I am a Help String
Additional help for built-in functions and operators is
available in the on-line version of the manual. Use the command
'doc <topic>' to search the manual index.
```

```
Help and information about Octave is also available on the WWW
at http://www.octave.org and via the help@octave.org
mailing list.
```

```
octave:2> help hello_octave_1
'hello_octave_1' is a function from the file /home/r00t/Writings/new/octave/
parts/codes/hello_octave_1.oct
```

```
I am a Help String
Additional help for built-in functions and operators is
available in the on-line version of the manual. Use the command
'doc <topic>' to search the manual index.
```

```
Help and information about Octave is also available on the WWW
at http://www.octave.org and via the help@octave.org
mailing list.
```

```
octave:3> hello_octave_1()
Hello from the second function! function has 0 input arguments and 0 output
arguments.
```

When we call the function `hello_octave_1()`, Octave looks for a file named `hello_octave_1.oct` and finds the symbolic link to the `hello_octave.oct` file. Octave is thus able to find the function and execute it.